

Módulo 4 – Testes Automatizados de API

Nome: Mariana Caetano Costa

Email: mariana.cae.costa@gmail.com

Telefone: (31) 9.8859-0581

OBJETIVO DO MÓDULO

Compreender como testar APIs REST de forma estruturada, aplicar a heurística VADER na identificação de cenários de teste e automatizar os principais fluxos utilizando JavaScript, Mocha, SuperTest e Chai.

1. PREPARAÇÃO DO AMBIENTE

Durante o módulo, o ambiente utilizado para os testes foi composto pelos seguintes recursos:

Banco API

- Git Bash: acesso à pasta **banco-api**
- Inicialização do projeto pelo comando: **npm run rest-api**
- Acesso à aplicação pelo navegador.

Banco Web

- Git Bash: acesso à pasta **banco-web**
- Inicialização do servidor pelo comando: **npm run server**
- Acesso ao sistema pelo navegador.

Banco de Dados MySQL

- Banco de dados utilizado: banco.
 - Usuário: julio.lima (acesso total)
 - Usuário: junior.lima (acesso limitado)
-

2. MENTALIDADE DE TESTES DE APIs

Testar APIs exige uma mudança de perspectiva em relação aos testes realizados na interface gráfica. Em vez de concentrar a validação em telas, campos e interações visuais, o QA analisa a comunicação entre os sistemas, observando endpoints, requisições, respostas, payloads JSON, headers, tokens e códigos de status HTTP.

Diferença entre testes de UI e testes de API

Testes de UI	Testes de API
Formulários	Endpoints
Telas	Métodos HTTP
Fluxos de navegação	Requisições e respostas
Interações visuais	Payloads JSON
Campos	Parâmetros
Elementos da interface	Autenticação e autorização
Comportamento visual	Integração entre serviços

A validação das regras de negócio continua sendo fundamental nos testes de API, porém passa a ser analisada juntamente com os aspectos técnicos da comunicação entre os serviços.

3. HEURÍSTICA VADER

A heurística VADER foi utilizada como guia para organizar a análise dos principais pontos que devem ser considerados durante os testes de uma API REST.

Letra	Elemento	Aspectos analisados
V	Verbs	Métodos HTTP e comportamento esperado
A	Authorization	Autenticação, autorização, tokens e permissões
D	Data	Tipos, formatos, valores, tamanho e paginação
E	Errors	Códigos HTTP, mensagens e tratamento de erros
R	Responsiveness	Tempo de resposta, timeout e concorrência

A aplicação da heurística permitiu organizar a investigação dos endpoints antes da definição dos cenários de automação.

3.1 V – Verbs

Foram analisados os principais métodos HTTP utilizados pela API:

Login

- POST – realização do login e geração do JWT.

Transferências

- POST – criação de transferência.
- GET – consulta das transferências.
- GET por ID – consulta de uma transferência específica.
- PUT – atualização completa de uma transferência.

- PATCH – atualização parcial.
- DELETE – exclusão de uma transferência.

Contas

- GET – consulta da lista de contas.
- GET por ID – consulta dos detalhes de uma conta específica.

Também foi considerada a utilização de métodos HTTP não permitidos, verificando se a API apresentou o comportamento esperado.

3.2 A – Authentication e Authorization

Autenticação – Quem é você?

É o processo de verificação da identidade do usuário ou sistema.

Autorização – O que você pode acessar ou fazer?

É a verificação das permissões disponíveis após a autenticação.

Foram considerados cenários como:

- token válido;
- token inválido;
- token expirado;
- ausência de token;
- usuário sem permissão;
- tentativa de acesso a recurso de outro usuário;
- tentativa de acesso a dados de outro tenant.

3.3 D – Data

Foram analisados os dados enviados e retornados pela API.

Exemplo de payload:

```
{  
  "contaOrigem": 0,  
  "contaDestino": 0,  
  "valor": 0,  
  "token": "string"  
}
```

A análise dos dados considerou:

- campos obrigatórios e opcionais;
- tipos de dados;
- formatos;
- valores permitidos e inválidos;
- limites;
- campos vazios ou ausentes;
- tamanho dos dados;
- estrutura do JSON;
- quantidade de registros;
- paginação.

3.4 E – Errors

A validação dos erros considerou os códigos HTTP definidos pelo contrato da API.

Código	Situação analisada
200	Operação realizada com sucesso
201	Recurso criado com sucesso
204	Operação concluída sem conteúdo na resposta
400	Parâmetros inválidos
401	Autenticação necessária ou inválida
403	Acesso não permitido
405	Método não permitido
422	Erro de validação dos dados
500	Erro interno do servidor

Os códigos efetivamente esperados devem estar de acordo com o contrato da API.

Também foi considerada a validação do corpo das respostas de erro, incluindo mensagens, estrutura dos dados e ausência de informações internas ou sensíveis.

Exemplo:

```
{  
  "message": "Conta não encontrada"  
}
```

3.5 R – Responsiveness

A análise de responsividade considerou o tempo necessário para que a aplicação respondesse às requisições.

O requisito analisado para o login estabeleceu tempo de resposta inferior a 50 milissegundos com 10 usuários conectados e enviando requisições durante 1 minuto.

Para essa validação, foi considerada a necessidade de execução de testes de performance utilizando o Postman.

Também foram considerados:

- tempo de resposta;
- timeout;
- concorrência;
- comportamento de falha rápida (*fail fast*).

4. ANTI-PATTERNS E PONTOS DE AUDITORIA

Durante a análise da API, foram considerados alguns comportamentos que podem indicar problemas de implementação ou validação.

Uso incorreto dos verbos HTTP

Exemplos:

- utilização de GET para excluir recursos;
- utilização de POST em situações que exigem atualização.

Falhas de autenticação e autorização

Exemplos:

- acesso a recursos protegidos sem token;
- acesso a informações sem a permissão necessária.

Validação frágil

Dados inválidos ou incompletos podem ser aceitos sem o tratamento adequado.

Mensagens de erro genéricas ou inadequadas

Exemplo:

"Algo deu errado"

Também foi considerada a utilização inadequada de códigos HTTP, como retornar 200 para uma operação que apresentou falha.

Falhas de tratamento de exceções

Foi considerada a necessidade de evitar a exposição de informações internas, como stack traces, nas respostas destinadas ao cliente.

Problemas de performance

Foram considerados tempos de resposta elevados ou inconsistentes, principalmente em situações de carga e concorrência.

5. APLICAÇÃO DA HEURÍSTICA VADER NA AUTOMAÇÃO

A heurística VADER foi utilizada como referência para identificar os cenários que deveriam ser contemplados na automação.

Antes da criação dos testes automatizados, os endpoints foram analisados considerando:

V – Verbs: Validação dos métodos HTTP e do comportamento esperado.

A – Authorization: Validação de autenticação, tokens e permissões.

D – Data: Validação dos dados enviados e recebidos.

E – Errors: Validação dos códigos HTTP e mensagens de erro.

R – Responsiveness: Avaliação do comportamento da API em relação ao tempo de resposta, timeout e concorrência.

Dessa forma, a VADER foi utilizada como guia de investigação e organização dos cenários de teste.

6. AUTOMAÇÃO DE API REST

A automação dos testes foi estruturada utilizando as seguintes bibliotecas:

Mocha: Framework utilizado para organização e execução dos testes.

SuperTest: Biblioteca utilizada para realizar requisições HTTP e testar endpoints.

Chai: Biblioteca utilizada para realizar as asserções dos resultados obtidos.

A estrutura utilizada relacionou as ferramentas da seguinte forma:

Mocha → organização e execução dos testes

SuperTest → requisições HTTP

Chai → validação dos resultados

6.1 Estrutura do projeto

A estrutura do projeto foi organizada da seguinte maneira:

```
banco-api-tests
├── test
│   └── Login.test.js
```

6.2 Inicialização do projeto

O projeto foi inicializado pelo Git Bash com: `npm init -y`

As bibliotecas necessárias foram instaladas com: `npm install -D mocha supertest chai`

6.3 Script de teste de Login

Foi estruturado o primeiro script de teste para o endpoint de login da API, utilizando Mocha, SuperTest e Chai.

6.4 Execução dos testes

O Mocha foi configurado no package.json para localizar os arquivos de teste:

```
"scripts": { "test": "mocha ./test/**/*.test.js --timeout=200000" }
```

A execução da suíte foi realizada pelo comando: `npm test`

7. GERAÇÃO DE RELATÓRIOS

- Foi utilizada a biblioteca **Mochawesome** para geração dos relatórios de execução dos testes.
 - Instalação: `npm install -D mocha chai mochawesome`
 - O script de execução foi configurado com o reporter:

```
"scripts": { "test": "mocha ./test/**/*.test.js --timeout=200000 --reporter mochawesome" }
```
 - A pasta de relatórios foi tratada como um artefato temporário e adicionada ao `.gitignore`.
 - Também foi criada a branch: `relatorio-execucao-testes`
 - O fluxo de versionamento contemplou o envio das alterações ao GitHub e a integração com a branch principal.
-

8. SCRIPT DE TESTE PARA TRANSFERÊNCIAS

Os testes automatizados de transferência foram organizados como parte da aplicação prática dos conceitos apresentados no módulo, considerando os endpoints e as regras de negócio analisadas durante o estudo.

9. REFATORAÇÃO PARA MANUTENÇÃO

Variáveis de ambiente com dotenv

Documentação dotenv: <https://www.npmjs.com/package/dotenv>

Instalação: `npm install -D dotenv`

- Criar arquivo `.env` na raiz com dados como `BASE_URL`.
- Carregar as variáveis com `import 'dotenv/config'`.
- Substituir URLs fixas por `process.env.BASE_URL`, facilitando troca de ambientes.

10. MELHORANDO PROJETO DE TESTES AUTOMATIZADOS HELPERS

Funções de suporte aos testes

Criar a pasta helpers

- Criar o arquivo `autenticação.js`
- Criar uma função anônima passando por parâmetro (usuário e senha)
- Copiar a função de token da transferência e colar aqui, nos dados de usuário substituir pelos parâmetros.
- No teste de transferência importe o helper e autenticação
- Substitua a o trecho de obter token pela chamada da função passando os dados de login do usuário

11. MELHORANDO PROJETO DE TESTES AUTOMATIZADOS HOOK BEFORE EACH

Hook	Momento	Uso típico
before	Uma vez antes da suíte	Preparar ambiente
beforeEach	Antes de cada teste	Gerar token/massa
afterEach	Após cada teste	Limpar dados
after	Uma vez ao final	Encerrar recurso

12. MELHORANDO PROJETO DE TESTES AUTOMATIZADOS FIXTURES E MASSA DE DADOS

- Fixtures são arquivos estáticos, normalmente JSON, usados para centralizar massas de teste.
 - Para alterar somente um campo sem modificar o objeto original, pode-se usar cópia rasa com spread: `{ ...objeto }`.
 - Em objetos aninhados, uma cópia profunda como `structuredClone(objeto)` evita compartilhamento de referências internas.
-

13. VALIDANDO DADOS NO CORPO DA RESPOSTA

Implementado:

- Get /transferencia/{id}: trazendo registro específico.
- Get /transferência: utilizando limit e validando um vetor de objetos.

Além do status code, os testes devem validar o corpo da resposta, valores, tipos, quantidade de itens e, quando necessário, confirmar a persistência diretamente no banco de dados.

14. CRIANDO O README DO PROJETO

Utilizando a inteligência artificial para criar a documentação do projeto.

Tenho um projeto e automação de testes de API Rest disponível em <https://github.com/MarianaCaetanoCosta/Mentoria-2.0-JulioDeLima/tree/main/Projetos/banco-api-tests/2-supertest> que testa a API Rest do projeto <https://github.com/MarianaCaetanoCosta/Mentoria-2.0-JulioDeLima/tree/main/Projetos/banco-api>.

Meu projeto faz uso da linguagem de programação JavaScript e usa a biblioteca Mocha, Supertest, Chai, entre outras(listadas n package,json) para realizar a estruturação dos testes.

Há alguns documentos que não são listados no repositório mas que precisam de explicação também, por exemplo, o arquivo .env que possui uma variável chamada BASE_URL e que precisa ser criado pelo usuário do projeto. Além dele, o mochawesome, que gera um relatório em HTML no diretório de mesmo nome.

Preciso que crie o README para esse projeto, coloque o nome do projeto, seu objetivo, a stack utilizada, a estrutura de diretórios, o formato do arquivo .env, os comandos para execução do teste e obtenção de relatórios e links para acesso a documentação das dependências utilizadas.

Após a revisão, quero baixar o arquivo com o formato .md

15. SÍNTESE DO APRENDIZADO

O módulo abordou testes de APIs REST, incluindo métodos HTTP, autenticação, autorização, validação de dados, códigos de status, erros e responsividade. A heurística VADER foi utilizada para orientar a identificação e organização dos cenários de teste. Na prática, foram desenvolvidos testes automatizados com JavaScript, Mocha, SuperTest e Chai, incluindo validações de status, respostas, dados e paginação. Também foram aplicados conceitos de organização da automação, como dotenv, helpers, hooks, fixtures e geração de relatórios com Mochawesome, além do uso de Git e GitHub para versionamento.

MOCHA

Sintaxe / Recurso	Objetivo	Aplicação no projeto
describe()	Agrupar testes relacionados	Agrupar Contas, Login e Transferências
it()	Definir um caso de teste	Criar cada cenário de teste
before()	Executar uma preparação uma vez antes da suíte	Preparações que precisam ocorrer apenas uma vez
after()	Executar uma ação uma vez depois da suíte	Limpeza após os testes
beforeEach()	Executar uma preparação antes de cada teste	Obter um novo token antes de cada teste
afterEach()	Executar uma ação depois de cada teste	Limpeza de dados criados pelo teste
async/await	Trabalhar com operações assíncronas	Aguardar login e requisições HTTP
this.timeout()	Definir tempo máximo para execução	Aumentar timeout quando a API demora
.skip	Ignorar temporariamente um teste	it.skip(...)
.only	Executar somente um teste/suíte	it.only(...) durante uma investigação

SUPERTEST

Sintaxe / Recurso	Objetivo	Aplicação no projeto
request()	Criar uma requisição para a API	request(process.env.BASE_URL)
.get()	Realizar requisição GET	Consultar contas e transferências
.post()	Realizar requisição POST	Criar transferência e realizar login
.put()	Realizar requisição PUT	Atualizações completas, quando houver endpoint
.patch()	Realizar requisição PATCH	Atualizações parciais, quando houver endpoint
.delete()	Realizar requisição DELETE	Exclusão de registros, quando houver endpoint
.set()	Definir headers HTTP	Content-Type e Authorization
.send()	Enviar corpo da requisição	Enviar bodyTransferencias
.query()	Enviar parâmetros de consulta	Paginação e filtros
.expect(status)	Validar status HTTP diretamente	.expect(200)
.expect(header, valor)	Validar header da resposta	Validar Content-Type
.expect(body)	Validar corpo esperado	Validar resposta conhecida
.expect(fn)	Criar validação personalizada	Validações específicas da resposta

CHAI — ASSERTIONS

Sintaxe	Objetivo	Aplicação no projeto
<code>expect(X).to.equal(Y)</code>	Validar igualdade exata	<code>expect(id).to.equal(1)</code>
<code>expect(X).to.not.equal(Y)</code>	Validar que dois valores são diferentes	Garantir que um valor não seja o esperado
<code>expect(X).to.be.a('string')</code>	Validar tipo string	titular, saldo, valor quando retornados como string
<code>expect(X).to.be.a('number')</code>	Validar tipo number	id, conta_origem_id, conta_destino_id
<code>expect(X).to.be.a('boolean')</code>	Validar tipo boolean	Campos booleanos da API
<code>expect(X).to.be.an('array')</code>	Validar que é um array	contas, transferencias
<code>expect(X).to.be.an('object')</code>	Validar que é um objeto	resposta.body
<code>expect(X).to.have.property('campo')</code>	Validar existência de propriedade	Verificar se id, titular, saldo existem
<code>expect(X).to.have.property('campo', Y)</code>	Validar propriedade e valor	<code>property('id', 1)</code>
<code>expect(X).to.have.lengthOf(Y)</code>	Validar quantidade exata	10 transferências
<code>expect(X).to.be.at.most(Y)</code>	Validar quantidade/valor máximo	Até 10 contas
<code>expect(X).to.be.at.least(Y)</code>	Validar quantidade/valor mínimo	Pelo menos 1 registro
<code>expect(X).to.be.above(Y)</code>	Validar valor maior que outro	Valor acima do limite
<code>expect(X).to.be.below(Y)</code>	Validar valor menor que outro	Transferência abaixo de R\$ 10
<code>expect(X).to.be.within(A, B)</code>	Validar valor dentro de intervalo	Valores entre dois limites
<code>expect(X).to.include(Y)</code>	Validar inclusão de valor/conteúdo	Verificar se uma lista contém determinado elemento
<code>expect(X).to.be.true</code>	Validar true	Campo booleano verdadeiro
<code>expect(X).to.be.false</code>	Validar false	Campo booleano falso
<code>expect(X).to.be.null</code>	Validar null	Campo que deve ser nulo
<code>expect(X).to.be.undefined</code>	Validar undefined	Campo que não deveria estar definido
<code>expect(X).to.exist</code>	Validar existência do valor	Garantir que token ou campo exista